Selección de variables

2da parte

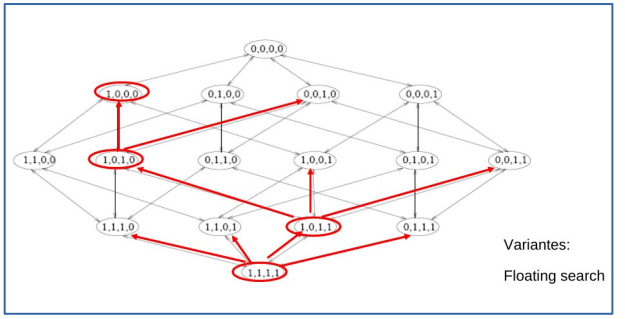
Repaso

- Métodos de filtro
  - Uso de un criterio diferente al modelo final
  - Generalmente univariado
  - Rápidos (en general O(p))
- Wrappers
  - Buscar la mejor combinación con el modelo
  - Multivariado
  - Heurísticas de búsqueda. Back y For.

Métodos embebidos

- Los wrappers backward son potencialmente los mejores métodos de selección.
- Son computacionalmente muy pesados.
  - A cada paso construyen todos los clasificadores intermedios posibles para evaluarlos.
  - Para rankear p variables crean $O(p^2)$ modelos.
- La solución ideal sería un método backward, basado directamente en el modelo final, pero eficiente.

Métodos embebidos



- Para evaluar cuál es la próxima variable a eliminar, el wrapper back construye todos los modelos con una variable menos.
- Los evalúa a todos y “da un paso” en la dirección de máximo descenso del error.
- Se puede hacer algo parecido sin calcular todos los modelos target?

Métodos embebidos

- Lo que necesitamos conocer es la derivada del error respecto de cada variable.
  - O alguna aproximación a la derivada
  - Se las llama “medidas internas de importancia”
- Si la función error es razonablemente suave, dar el paso en la dirección de máximo descenso de la derivada debería ser lo mismo que el máximo descenso del error.

Métodos embebidos

- Recursive Feature Elimination (RFE):
  - Ajustar un modelo a los datos
  - Rankear las variables usando una medida interna de importancia.
    - Más importante es la que más empeora al modelo al ser eliminada
  - Eliminar la variable (o un grupo) con el ranking más bajo
  - iterar

El algoritmo RFE

Inputs:

Training set T

Set of p features $F = \{f_1, ..., f_p\}$

Ranking method $M(T, F)$

Outputs:

Final ranking R

Code:

Repeat for i in $\{1 : p\}$

Rank set F using $M(T, F)$

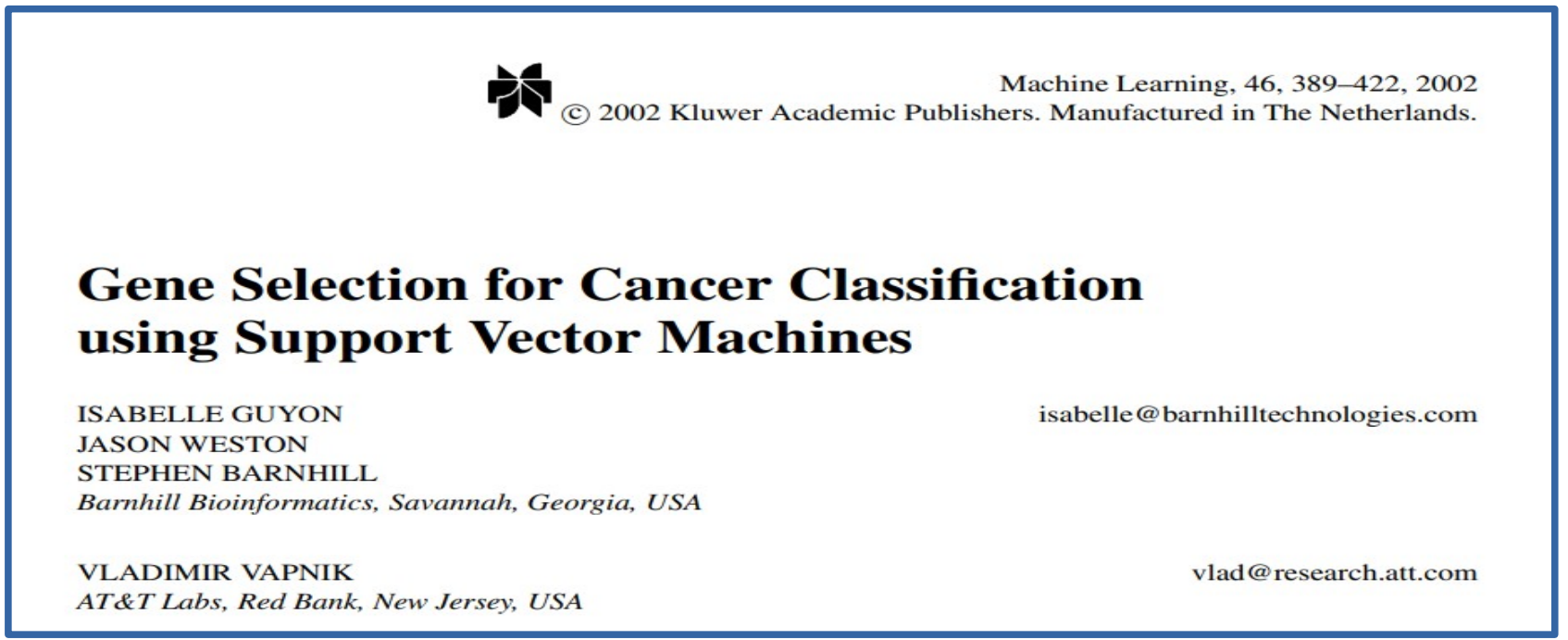
$f^* \leftarrow$ last ranked feature in F

$R(p - i + 1) \leftarrow f^*$

$F \leftarrow F - f^*$

Fig. 1. Pseudo-code for the Recursive Feature Elimination (RFE) algorithm.

RFE con SVM



Machine Learning, 46, 389–422, 2002
© 2002 Kluwer Academic Publishers. Manufactured in The Netherlands.

Gene Selection for Cancer Classification using Support Vector Machines

ISABELLE GUYON isabelle@barnhilltechnologies.com
JASON WESTON
STEPHEN BARNHILL
Barnhill Bioinformatics, Savannah, Georgia, USA

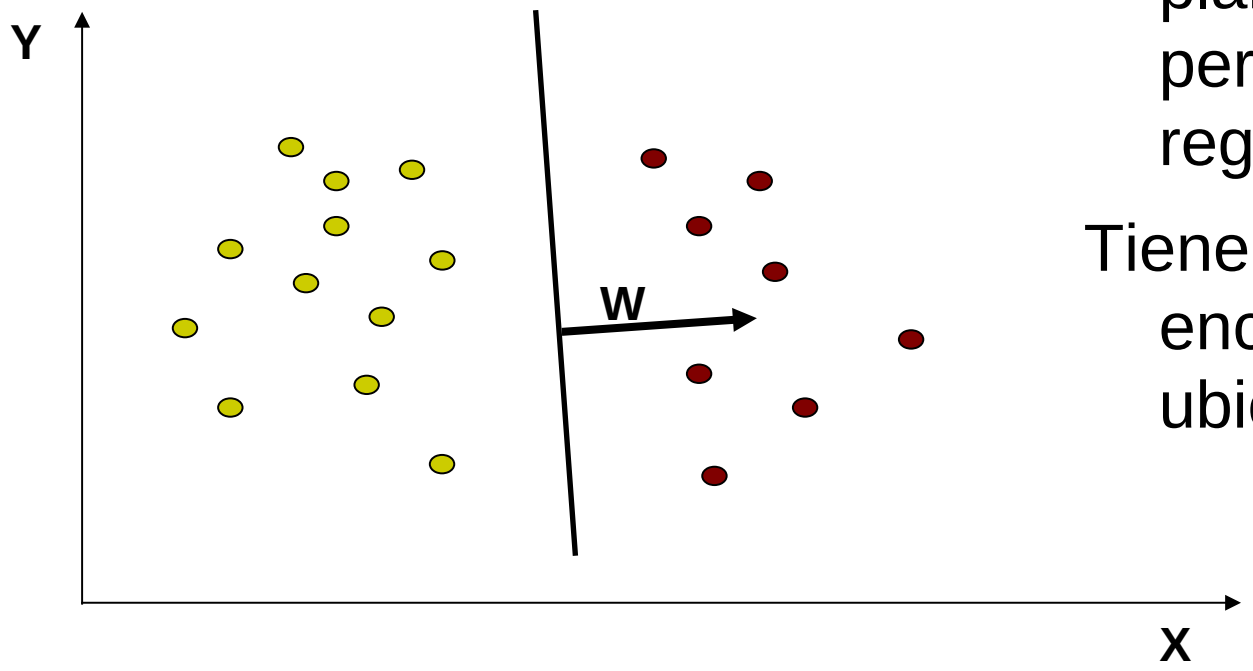
VLADIMIR VAPNIK vlad@research.att.com
AT&T Labs, Red Bank, New Jersey, USA

12900 citas 9/25

Isabelle Guyon home page

RFE con SVM

Medida de importancia SVM

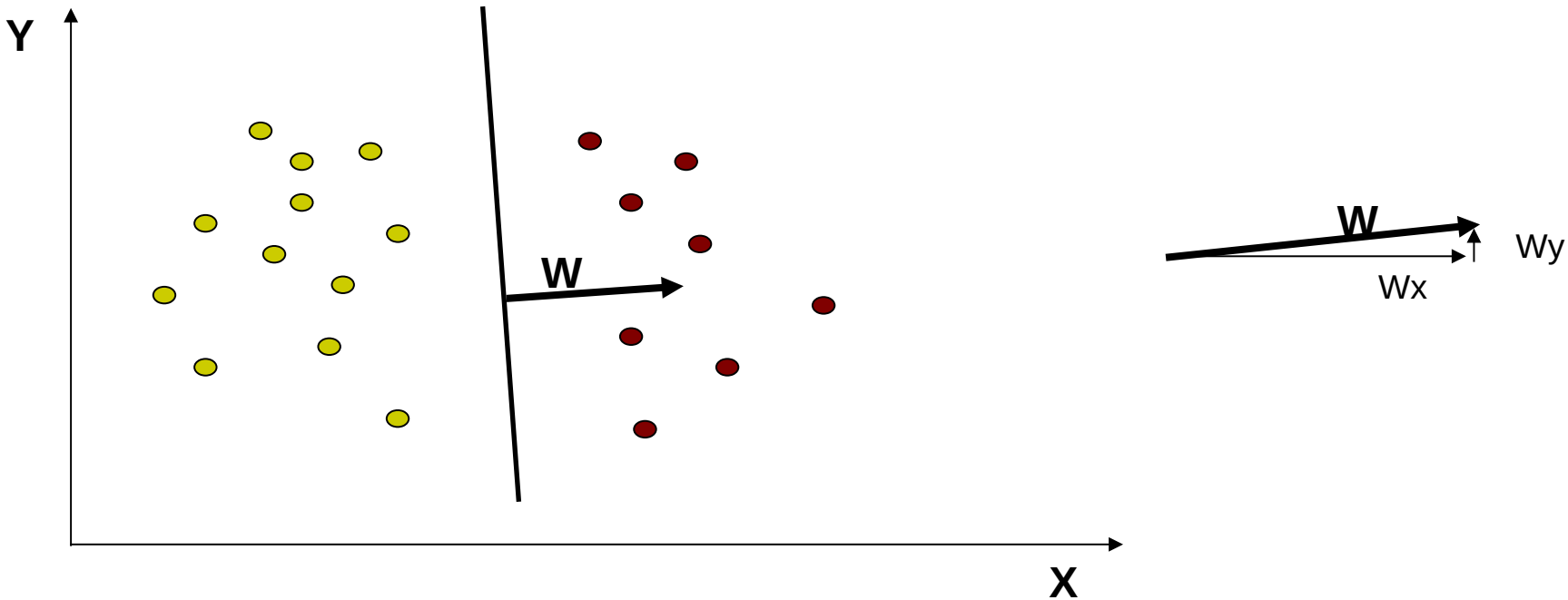


Las SVM usan un hiper plano similar a un perceptron con la regla del signo

Tienen otra forma de encontrar la ubicación

RFE con SVM

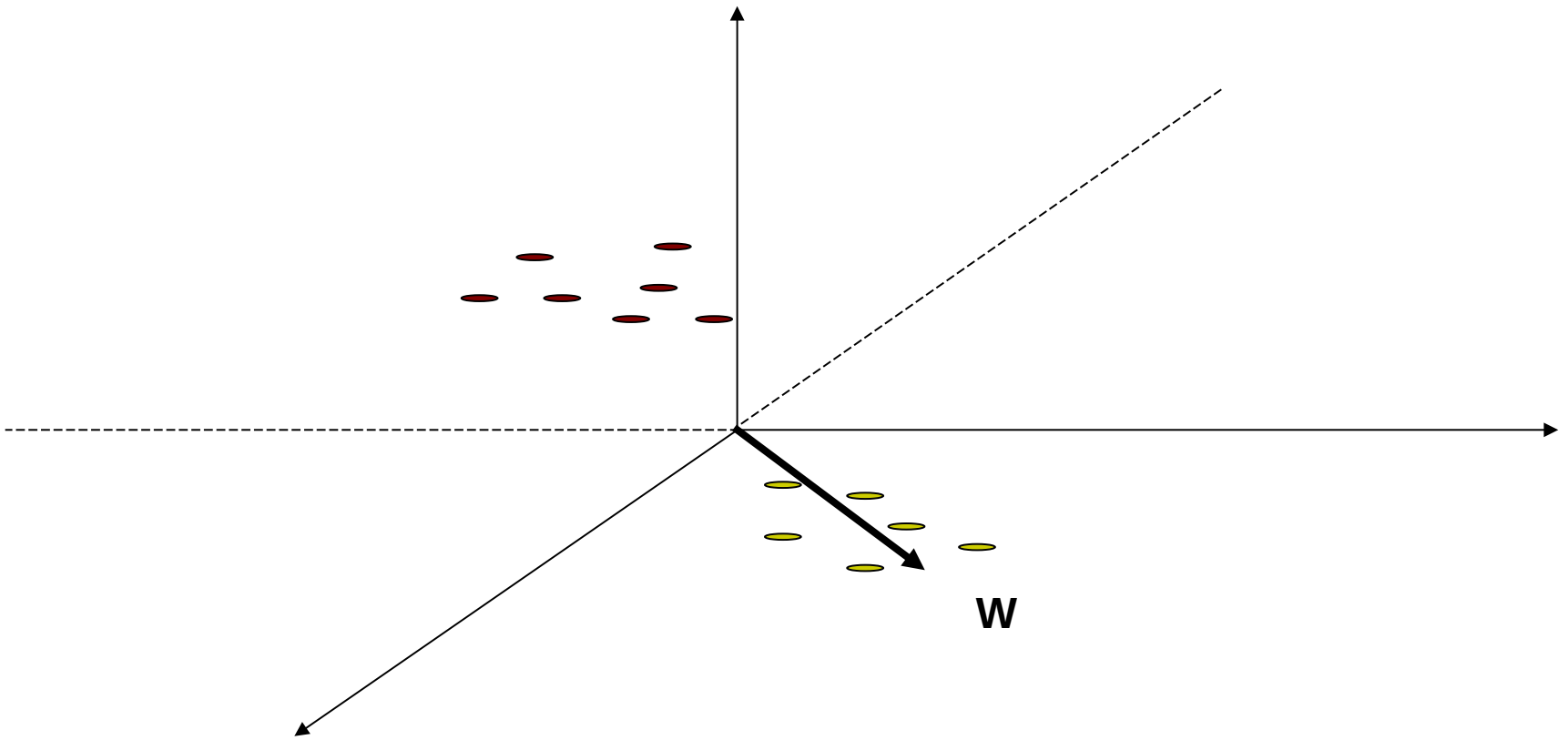
Medida de importancia SVM: componentes de W



Medidas de importancia

- Ejemplos de medidas de importancia:
  - SVM: componentes de W
  - Random Forest:
    - Shuffling OOB
    - Variación del GINI index.
  - LDA o PDA o Regresión logística: weights $\longrightarrow C(x) = \sigma\left(\sum_i w_i x_i\right)$
  - Partial Least Squares (PLS): Scores

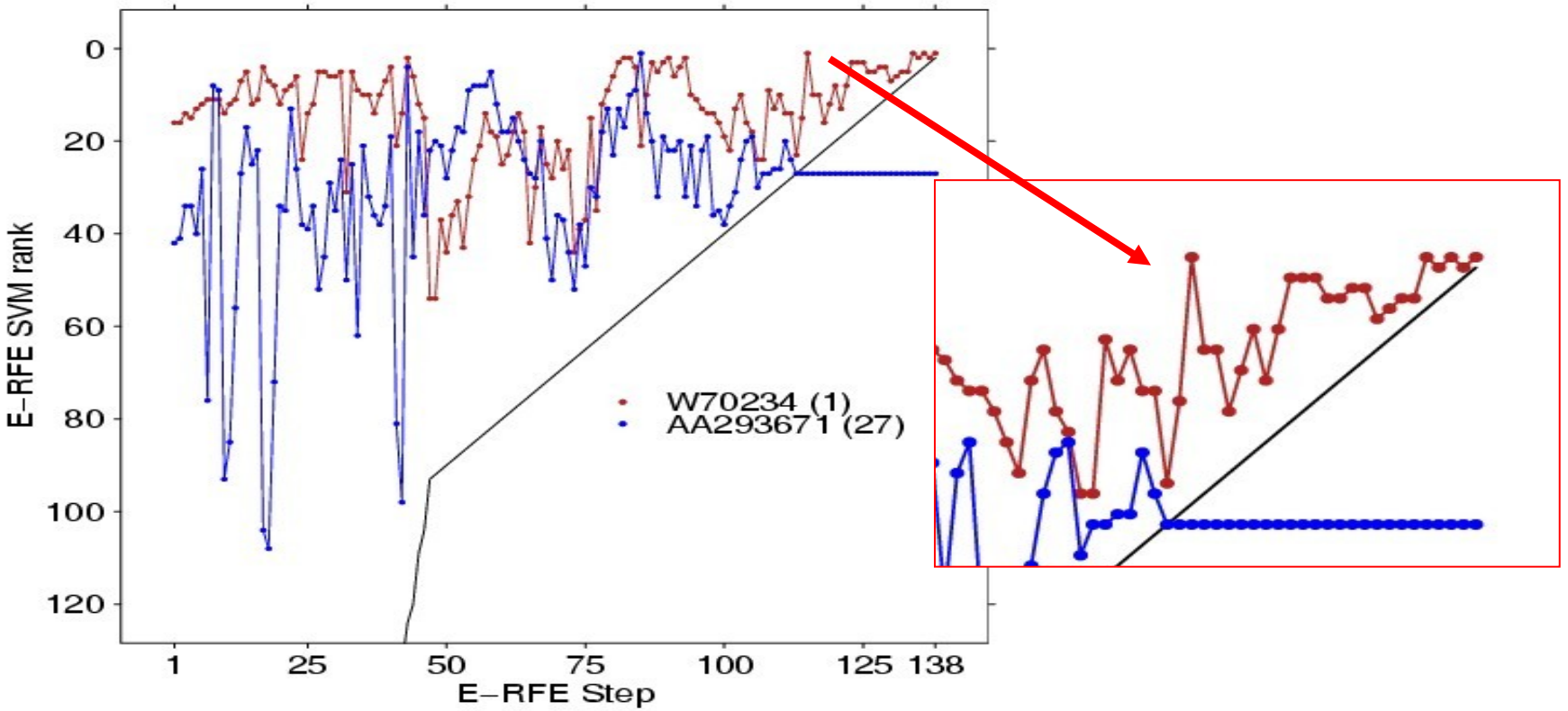
Problemas con el RFE



Problemas con el RFE

- Variables importantes pero correlacionadas
  - Si el modelo original usa las dos, las dos “comparten” la importancia.
  - En la práctica aparecen como menos importantes que otras variables.
  - Al eliminar una de ellas, la otra toma toda la importancia y suele subir bruscamente en el ranking (otra razón por la que es importante iterar).

Problemas con el RFE



(d) Two highly correlated features.

Problemas con el RFE

- Variables importantes pero correlacionadas
  - Cuál es eliminada y cuál promocionada es casi chance.
  - Como resultado, el ranking de variables tiende a ser inestable.

Resumen: Métodos Embebidos o de Ranking

- Pros:
  - Rápidos
  - Efectivos
  - Entendibles
  - Más estables que los wrappers greedy
- Contras:
  - La importancia de las variables se estima, no se mide directamente
  - Problemas de inestabilidad con variables correlacionadas

Otras variantes

- Algunos métodos de aprendizaje incorporan la selección directamente.
  - Ejemplo: Arboles de decisión (c4.5) hacen una selección greedy forward.
- Suelen no ser muy eficientes
- Hay excepciones. En regresión, los métodos “lasso” realizan una regresión lineal utilizando solo las variables necesarias

Otras variantes (no las vemos)

- En vez de seleccionar variables (problema combinatorio), por que no darle un peso estadístico a cada variable?
- Muchos métodos. La mayoría basado en el wrapper approach.
- Receta: se optimizan los pesos junto con todos los otros parámetros del método de aprendizaje.

Evaluando las selecciones

- Quiero saber cuántas variables me tengo que quedar
- Dos caminos:
  - Quedarme con todas las variables útiles
  - Quedarme sólo con las necesarias

Útiles o necesarias?

- Útil: cualquier variable que tiene información ***relevante*** para el modelado
- Necesaria: cualquier variable que tiene información ***indispensable*** para el modelado

- Todas las variables necesarias son útiles, pero no al revés
- Una variable puede ser útil en presencia de otras con las que comparte información, pero se puede transformar en necesaria cuando reduzco el conjunto de variables.

Evaluando las selecciones

- Quedarme con todas las variables útiles

- Cómo puedo saber si una variable es útil?

Evaluando las selecciones

- Quedarme con todas las variables útiles

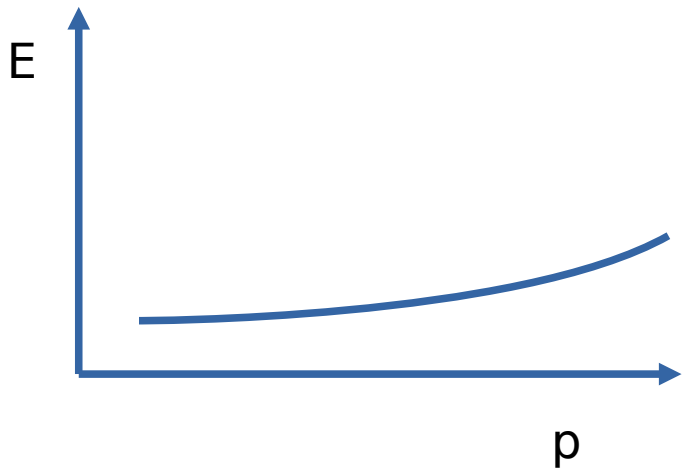
- Cómo puedo saber si una variable es útil?

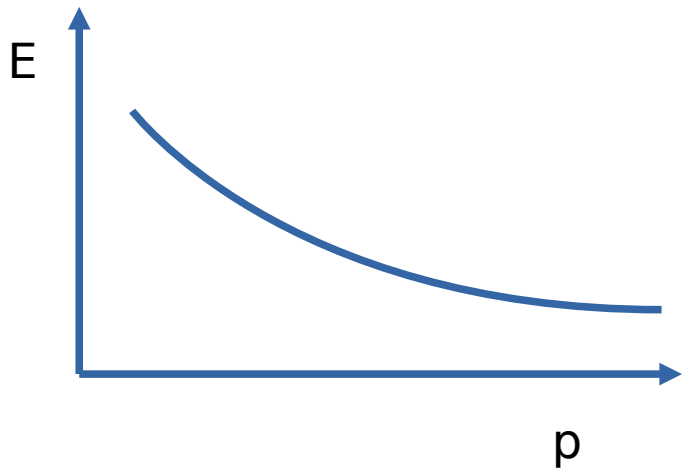
  La comparo con ruido!

- Introduzco una variables artificial de ruido, y uso un método de ranking. Me quedo con las que rankean mejor que el ruido.
  - Para más seguridad, agrego 10 y me quedo con todas las que son mejores que 9 de las agregadas

Evaluando las selecciones

- Quedarme sólo con las necesarias
- Es muy útil en general tener idea de cómo cambia el error de predicción al eliminar variables.





Evaluando las selecciones

- Es muy útil tener idea de cómo cambia el error de predicción al eliminar variables.
- Idea base:
  - Aplico el wrapper o filtro o RFE a mi problema.
  - Al ir eliminando variables, controlo el error. Busco el mínimo.
  - Qué error?

Evaluando las selecciones

- Idea base:
  - Aplico el wrapper o filtro o RFE a mi problema.
  - Al ir eliminando variables, controlo el error. Busco el mínimo.
  - Qué error? Conjuntos chicos. En principio se usó cross-validation
    - Divido los datos internamente.
    - Busco la variable a eliminar usando una parte de los datos, y calculo el error del modelo con otra parte de los datos. Repito y promedio (promedio el error y la selección).
    - Itero

Evaluando las selecciones

- Idea base:
  - Aplico el wrapper o filtro o RFE a mi problema.
  - Al ir eliminando variables, controlo el error. Busco el mínimo.
  - Qué error? Conjuntos chicos. En principio se usó cross-validation
    - Divido los datos internamente.
    - Busco la variable a eliminar usando una parte de los datos, y calculo el error del modelo con otra parte de los datos. Repito y promedio (promedio el error y la selección).
    - Itero

USO LOS DATOS DE TEST PARA ELEGIR EN OTRA PARTICION!!!

Evaluación incorrecta

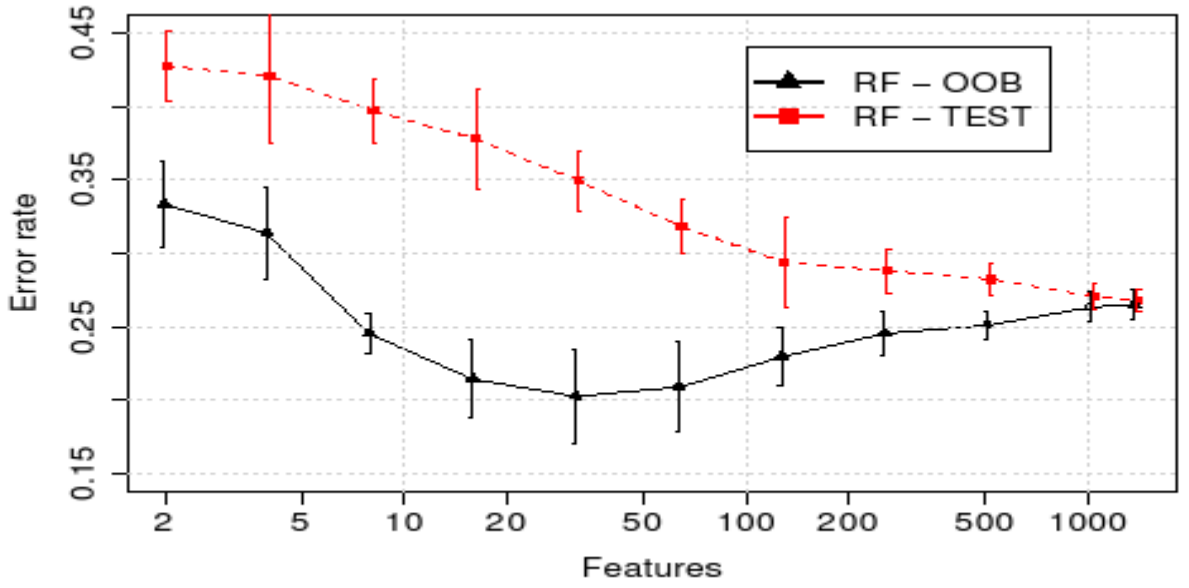


Figure 1: Error levels for OOB and real test set for RF-RFE on random data. We generated a full random dataset with 1380 random inputs with N (0, 1) distribution, with random labels, 131 of class 0 and 47 of class 1. Bayes error: 47/178=0,26. Error lines show one standard deviation.

Si buscamos mucho…

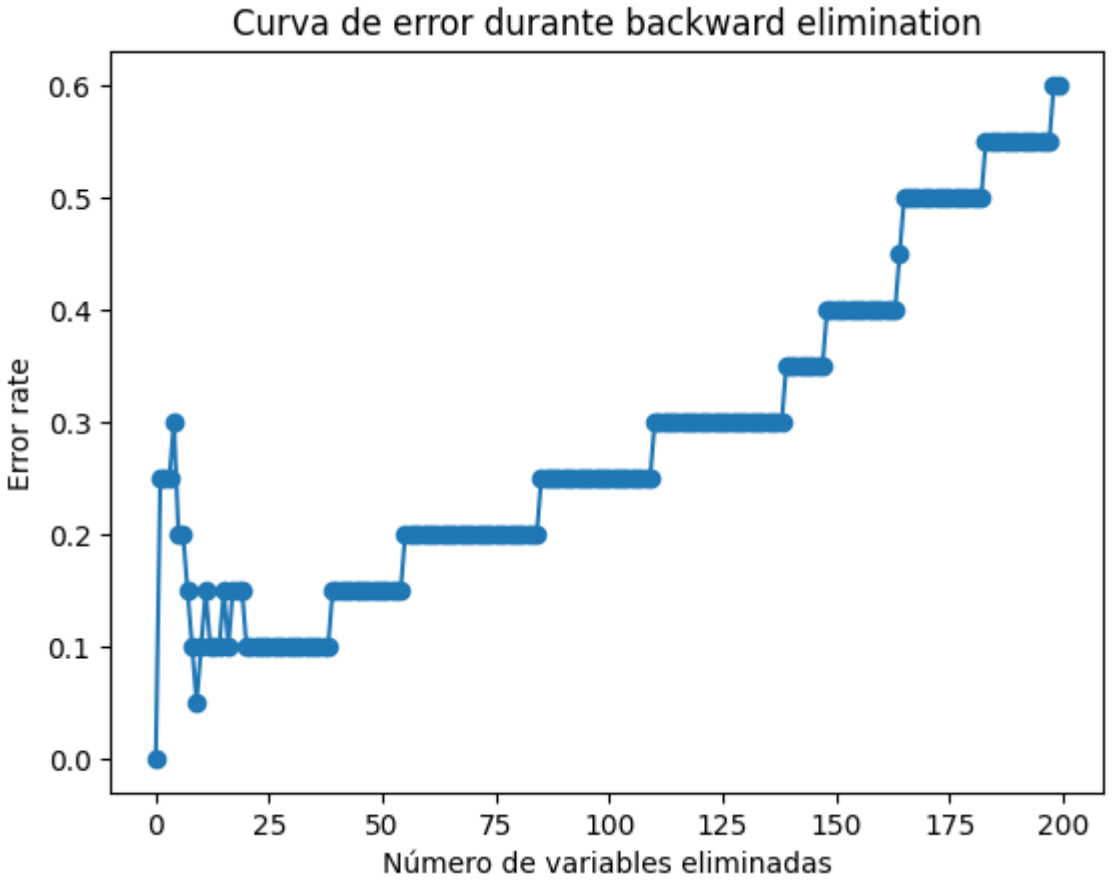
- Genero ruido, n=20, p=200 (Bayes=0.5)
- Los datos son ruido uniforme en todas las dimensiones
- Clasificador fijo, hiperplano apuntando al (1,1,...1)
- Busqueda greedy con cross-validation

```
while num_feat > 1:
        class_error = np.zeros(num_feat)
        for i in range(num_feat): # entrenar quitando la i-ésima
                mask = [j for k,j in enumerate(keep_feat) if k != i]
                x_train = X.iloc[:, mask]
                pred = np.sign(x_train.sum(axis=1))
                class_error[i] = error_rate(y, pred)

        best_index = np.argmin(class_error)
        list_feat[num_feat-1] = keep_feat[best_index]
        min_error[num_feat-1] = class_error[best_index]
```

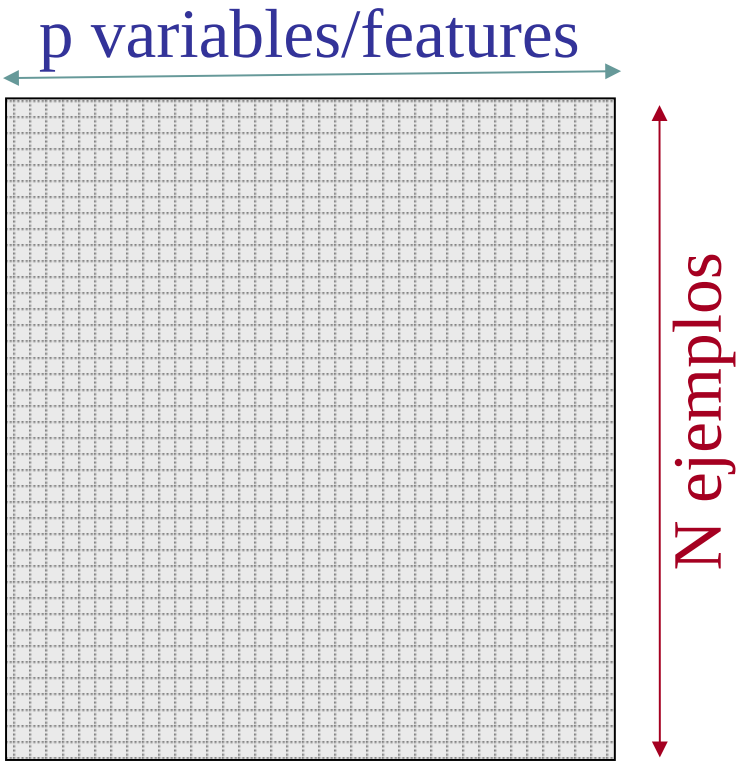
Código en notebook de demo

Si buscamos mucho…



- Error con 20 variables=0.1???
- Estoy haciendo tests múltiples!

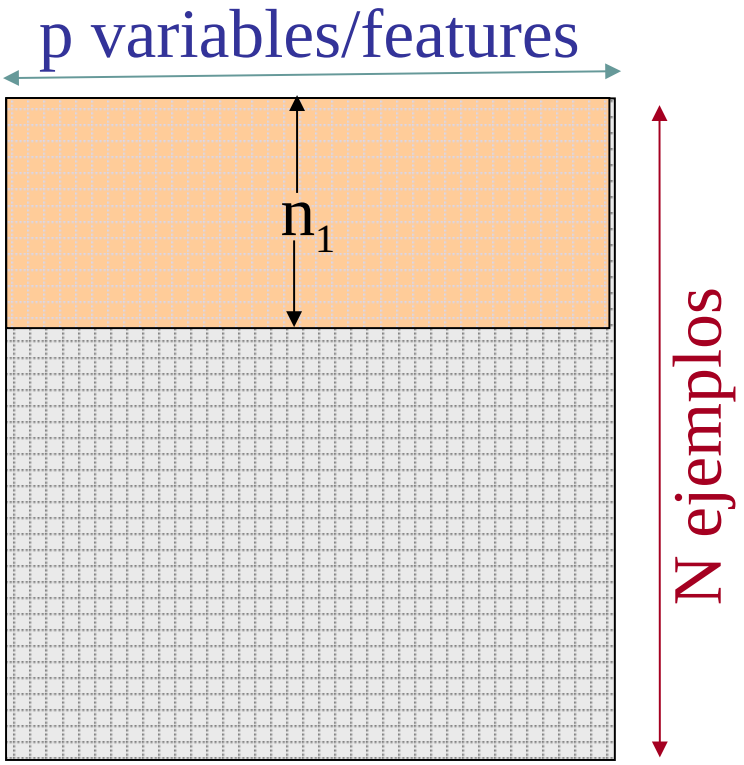
Evaluando las selecciones



Partir los datos en 3 partes:

training, validación y test set.

Evaluando las selecciones

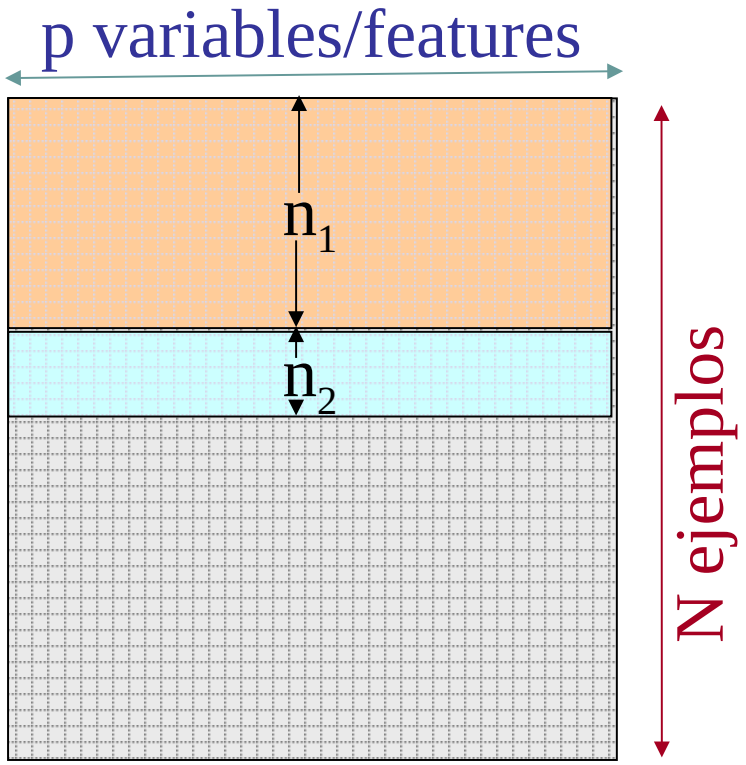


Partir los datos en 3 partes:

training, validación y test set.

1) Para cada subconjunto de variables entrenar un modelo en el training set.

Evaluando las selecciones



Partir los datos en 3 partes:

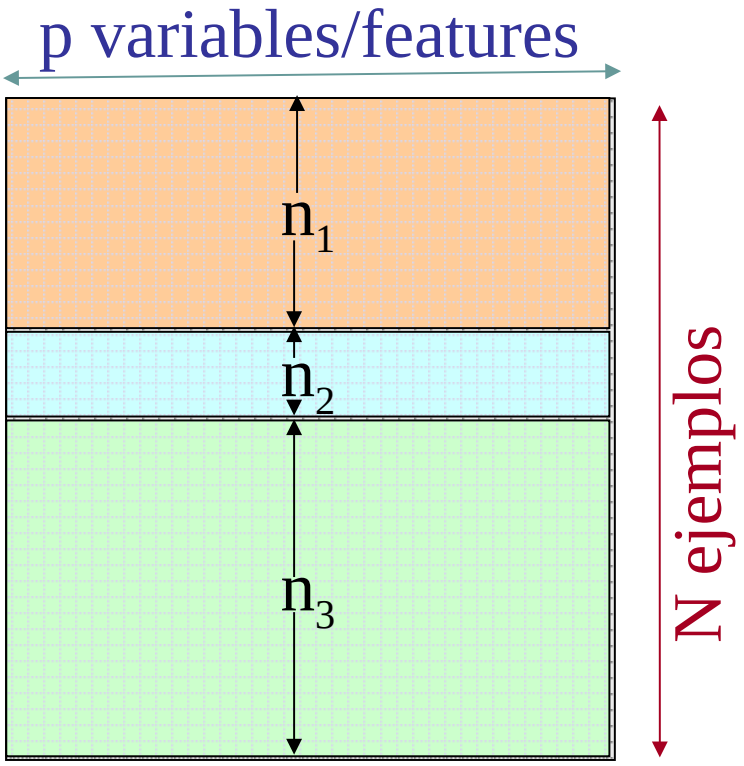
training, validación y test set.

1) Para cada subconjunto de variables entrenar un modelo en el training set.

2) Elegir el subconjunto de variables que muestra la mejor performance en el validation set.

- Se puede usar cross-validation para tener una mejor estima.

Evaluando las selecciones



Partir los datos en 3 partes:

training, validación y test set.

1) Para cada subconjunto de variables entrenar un modelo en el training set.
2) Elegir el subconjunto de variables que muestra la mejor performance en el validation set.

- Se puede usar cross-validation para tener una mejor estima.

3) Medir el error verdadero en el test set.

Complejidad en la selección

Con alta probabilidad (Vapnik):

$$\text{Generalization_error} \leq \text{Validation_error} + \varepsilon(C/n_2)$$

Generalization_error: Error de test, error real

Validation_error: Error estimado a partir de mi muestra

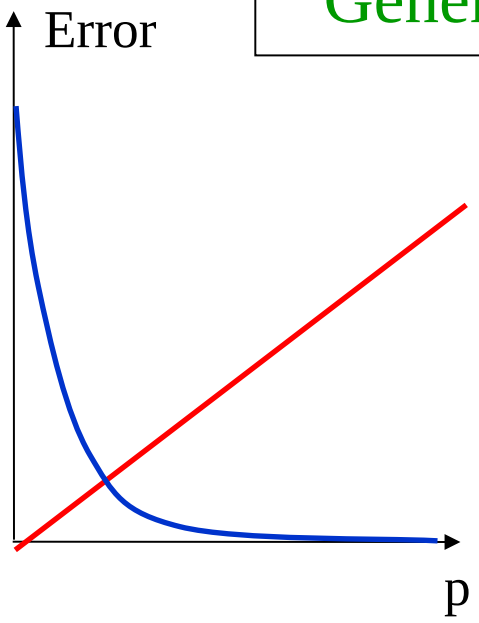
C: Complejidad de la busqueda

n_2: cantidad de ejemplos donde busqué

Complejidad en la selección

Con alta probabilidad (Vapnik):

$$\text{Generalization_error} \leq \text{Validation_error} + \varepsilon(C/n_2)$$



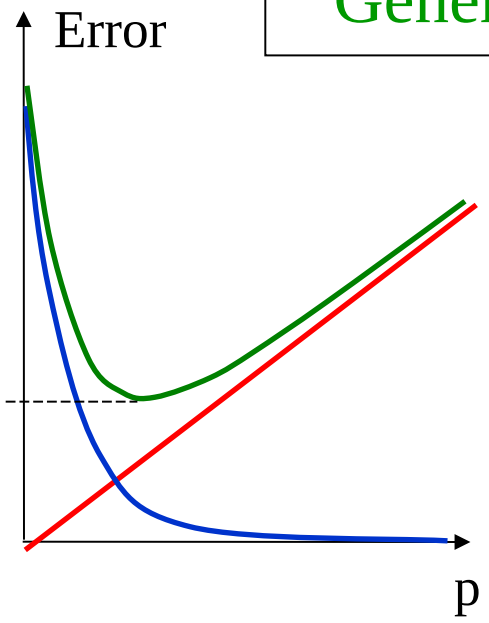
Método	Número de evaluaciones	Complejidad C
Búsqueda exhaustiva	2^P	P
Selecciones anidadas "ranking"	P(P+1)/2 o P	log P

n_2: número de ejemplos de *entrenamiento*, P: número total de variables, p: tamaño del subconjunto.

Complejidad en la selección

Con alta probabilidad (Vapnik):

$$\text{Generalization_error} \leq \text{Validation_error} + \varepsilon(C/n_2)$$



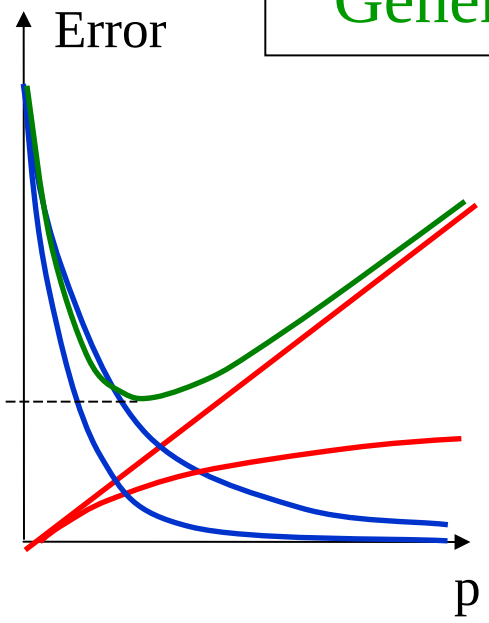
Método	Número de evaluaciones	Complejidad C
Búsqueda exhaustiva	2^P	P
Selecciones anidadas "ranking"	P(P+1)/2 o P	log P

n_2: número de ejemplos de *entrenamiento*, P: número total de variables, p: tamaño del subconjunto.

Complejidad en la selección

Con alta probabilidad (Vapnik):

$$\text{Generalization_error} \leq \text{Validation_error} + \varepsilon(C/n_2)$$



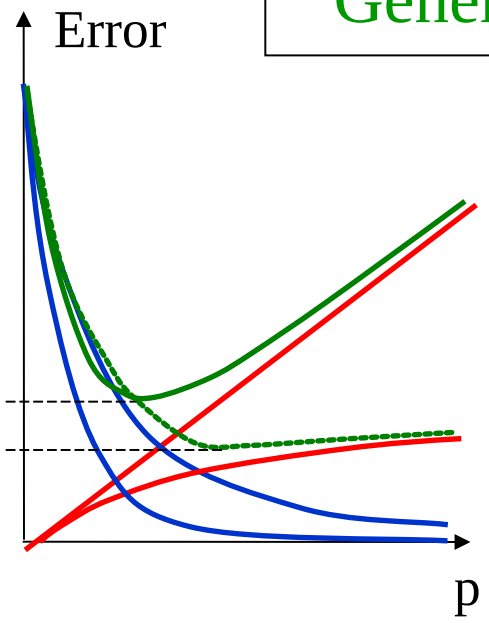
Método	Número de evaluaciones	Complejidad C
Búsqueda exhaustiva	2^P	P
Selecciones anidadas "ranking"	P(P+1)/2 o P	log P

n_2: número de ejemplos de *entrenamiento*, P: número total de variables, p: tamaño del subconjunto.

Complejidad en la selección

Con alta probabilidad (Vapnik):

$$\text{Generalization_error} \leq \text{Validation_error} + \varepsilon(C/n_2)$$



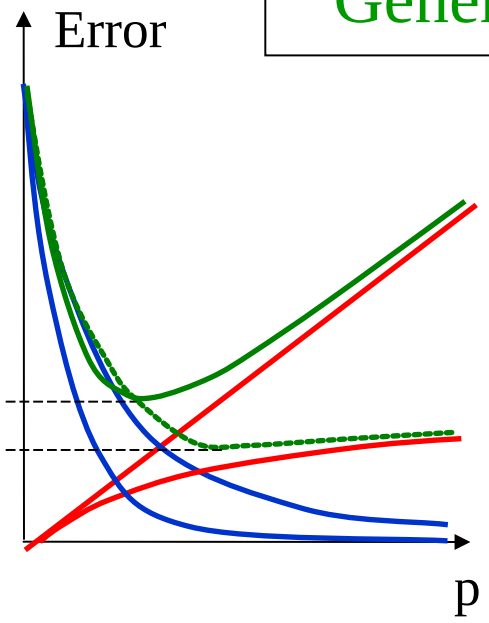
Método	Número de evaluaciones	Complejidad C
Búsqueda exhaustiva	2^P	P
Selecciones anidadas "ranking"	P(P+1)/2 o P	log P

n_2: número de ejemplos de *entrenamiento*, P: número total de variables, p: tamaño del subconjunto.

Complejidad en la selección

Con alta probabilidad (Vapnik):

$$\text{Generalization_error} \leq \text{Validation_error} + \varepsilon(C/n_2)$$



Método	Número de evaluaciones	Complejidad C
Búsqu… exhaustiva		
Selecciones anidadas "ranking"	P(P+1)/2 o P	log P

Acotar C mejora el resultado!

n_2: número de ejemplos de *entrenamiento*, P: número total de variables, p: tamaño del subconjunto.

Método sugerido para elegir y modelar

- Usar el train para elegir las variables.
- Usar validación independiente para determinar la cantidad de variables a retener.
- Hacer una selección final con train+validación.
- Estimar el error con el test set (pero no elegir nada).

Problema abierto

- Tenemos conjuntos chicos y necesitamos tests independientes. Usamos repeticiones para reducir la varianza de los resultados.

- Cada repetición da una lista distinta de variables. Cuáles son las variables importantes de verdad?

Problema abierto

- Tenemos conjuntos chicos y necesitamos tests independientes. Usamos repeticiones para reducir la varianza de los resultados.
- Cada repetición da una lista distinta de variables. Cuáles son las variables importantes de verdad?
  - Pensar cada lista como un votante. Buscar las más votadas
  - Tener en cuenta las “equivalentes” o “correlacionadas”, para eliminarlas o para reemplazar una por la otra

Práctico 2

- Armar el RFE, Backward Greedy y filter.
  - Hay funciones de importancia ya disponibles para los 3.
- Aplicar a algunos ejemplos artificiales y analizar los resultados.
- Dataset real: limpiarlo, graficarlo, dar información sobre variables relevantes